Developer Contribution & Project Learning Report

Aditya Gupta · Frontend Developer · Raga.ai

1. Employee Overview

Full Name	Aditya Gupta (12319077)
Role	Frontend Developer
Organization	Raga.ai
GitHub Profile	https://github.com/adityaragaai
Achievement	Pull Shark (GitHub Badge)
Report Period	Internship / Project Learning Phase — 2026

RagaAI is a product-based AI company that focuses on AI testing, evaluation, and observability. It was founded in 2022 by Gaurav Agarwal, a former NVIDIA and Ola tech leader. The company builds tools that help organizations test, monitor, and improve AI applications like Large Language Models, computer vision systems, and AI agents. Its key products include RagaAI Catalyst, Prism, and Canvas. The main goal of RagaAI is to make AI systems more reliable, safe, and production-ready.

2. Monthly Report — February 2026

Internship Report – FAB 2026 | Frontend SDE Intern Journey at RagaAI

I joined RagaAI as a Frontend SDE Intern in mid-February 2026. During the initial phase of my internship, I focused on understanding the organization, completing onboarding formalities, and getting familiar with the company's development workflow and technical ecosystem.

2.1 Organization Onboarding & HR Formalities

- Completed the onboarding process and organizational induction sessions.
- Finished mandatory HR formalities including POSH training and other compliance-related activities.
- Set up official accounts, communication tools, and development environment required for daily work.
- Understood company policies, work culture, and team collaboration processes.

2.2 Understanding the Organization & Projects

- Learned about the vision, products, and ongoing projects at RagaAI.
- Explored different teams and their responsibilities within the organization.
- Understood how frontend, backend, AI/ML, and DevOps teams collaborate during development.
- Gained knowledge about the various projects currently being developed in the organization.

2.3 Learning the Codebase

- Studied the existing frontend architecture and project structure.
- Learned how large-scale applications are organized and maintained.
- Explored reusable components, folder structures, API integrations, and state management patterns.
- Understood development standards and coding practices followed by the team.

2.4 Development Environment Setup

- Successfully cloned and configured multiple projects locally.
- Learned dependency management and project setup procedures.
- Configured environment variables and understood build/run workflows.
- Practiced debugging setup and project execution in local development environments.

2.5 Technology Stack Exploration

During the learning phase, explored the technologies used across projects, including:

Technology / Area	Details
React.js	Core frontend framework used across all projects
TypeScript	Strongly typed JS for scalable codebases
JavaScript	Base scripting and dynamic logic
Next.js	Server-side rendering and routing framework
Git & GitHub	Version control, branching, pull requests
REST APIs	Integration and consumption of backend services
Tailwind CSS / UI Libraries	Rapid, responsive UI development
Version Control & Branch Management	Feature branches, merging, and collaborative workflows

2.6 Version Control & Collaboration

- Learned the fundamentals of Git and version control workflows.

- Practiced cloning repositories, creating branches, committing changes, and pushing code.
- Understood collaborative development practices such as pull requests, code reviews, and branching strategies.
- Learned how teams manage feature development and bug fixes efficiently using Git workflows.

2.7 Key Learning Outcomes — February 2026

- Improved understanding of real-world frontend development workflows.
- Learned how enterprise-level projects are structured and managed.
- Developed familiarity with collaborative software development practices.
- Enhanced knowledge of frontend technologies and development tools.
- Built confidence in working with professional codebases and team environments.

3. Contributions — March to May 2026

The following section covers contributions made across three projects during March to May 2026.

ClinicalOps is a healthcare operations platform designed to streamline clinical workflows and improve communication between different teams. It helps manage clinical processes, user activities, and notifications through a centralized system. The platform focuses on making healthcare operations more efficient by reducing manual work and providing better visibility into ongoing tasks.

3.1 Clinical Ops UI

Period: March – May 2026

Contributed to the Clinical Ops UI — RagaAI's internal clinical operations platform. The work spanned two major modules: Fax Classification and the Collections Agent, both deeply integrated with backend APIs and built as part of the same Clinical Ops product.

3.1.1 Fax Classification Module

A core operational dashboard used by Super Admin to monitor and manage incoming fax classifications in real time.

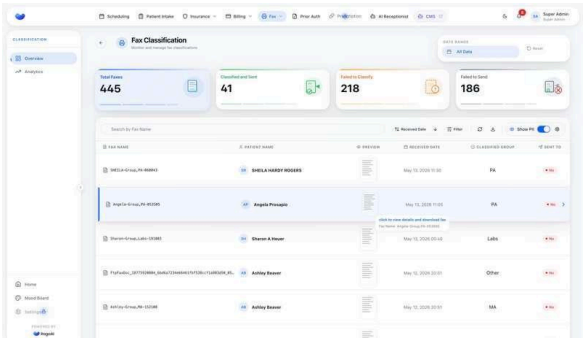
Link: https://clinicalops.ragaai.ai/admin/fax/classification

Key Contributions:

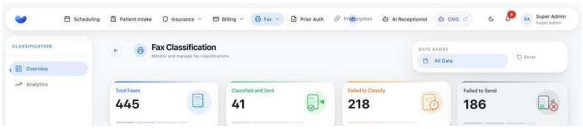
- Designed and built the Fax Classification Overview dashboard displaying live summary cards: Total Faxes (445), Classified and Sent (41), Failed to Classify (218), and Failed to Send (186).
- Integrated backend APIs to fetch and display real-time fax data including Fax Name, Patient Name, Preview, Received Date, Classified Group, and Sent To status per record.

- Implemented API response handling with proper loading, success, and error states across the fax list and summary metric cards.
- Added a CTA button for direct redirection to the CMS portal, enabling seamless navigation from Clinical Ops to the content management system.
- Built a Show PII toggle to conditionally display or hide personally identifiable information in the fax records table.
- Implemented fax search by name, date range filtering, and a download option for fax records.
- Added an interactive fax preview tooltip — clicking the preview icon shows fax name details and a download prompt inline.
- Ensured a responsive and accessible layout consistent with the RagaAI Clinical Ops design system.

Screens Developed:



Fax Classification — Overview Dashboard



Fax Classification — Header with CMS Portal CTA

3.1.2 Collections Agent — Billing Module (Advanced)

The Collections Agent is one of the most complex and high-value modules built during this internship. It is a fully integrated, AI-assisted billing and collections management system for Denver Allergy & Asthma Associates, combining multiple live API integrations, real-time Stripe payment data, ECW sync, and automated clinical workflows — all in a single unified interface.

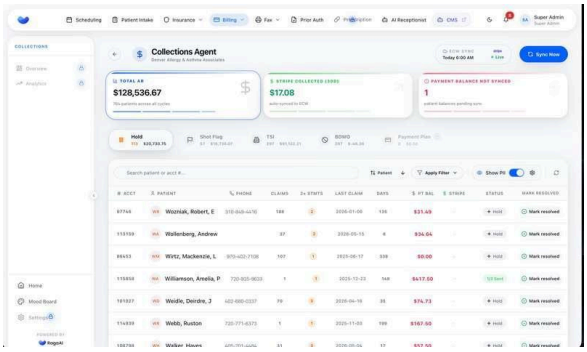
Link: https://clinicalops.ragaai.ai/admin/billing/collections

Key Contributions:

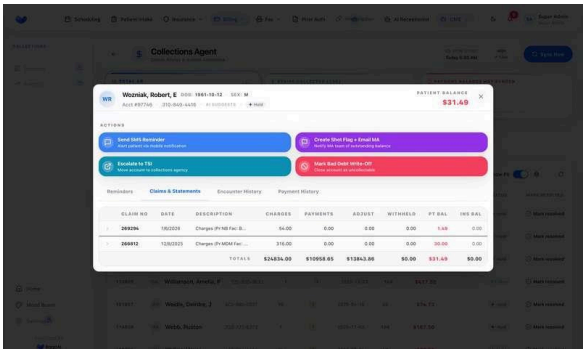
- Built the Collections Agent Overview dashboard displaying Total AR ($128,536.67 across 764 patients), Stripe Collected (30D), and Payment Balance Not Synced — all pulling from live APIs.
- Implemented category tabs — Hold, Shot Flag, TSI, BDWO, and Payment Plan — each with live patient counts and dollar amounts fetched from the backend.
- Integrated ECW Sync and Stripe Live status indicators with a Sync Now trigger button for real-time data reconciliation.

- Built the Patient Detail Dialog with 4 tabbed views: Reminders, Claims & Statements, Encounter History, and Payment History — all populated via separate API calls per tab.
- Implemented 4 fully functional action buttons inside the Patient Dialog, each triggering a dedicated API call: Send SMS Reminder (patient mobile notification), Create Shot Flag + Email MA (notifies MA team with a PDF shot flag document), Escalate to TSI (moves account to collections agency), and Mark Bad Debt Write-Off (closes account as uncollectable).
- Successfully integrated the Email Communications flow — built a live render pipeline UI showing a real-time email preview sent from notifications@ragaai.ai to ma_team@denverallergy.com with an auto-generated shot_flag PDF attachment.
- Verified end-to-end email delivery — the shot flag email with PDF attachment was successfully received in Gmail, confirming the full API pipeline works correctly.
- Implemented paginated patient list with configurable rows per page, showing stripe collected totals, page balance, and total balance in the footer.
- Added Show PII toggle, search by patient or account number, Apply Filter, column sort, and data export controls.

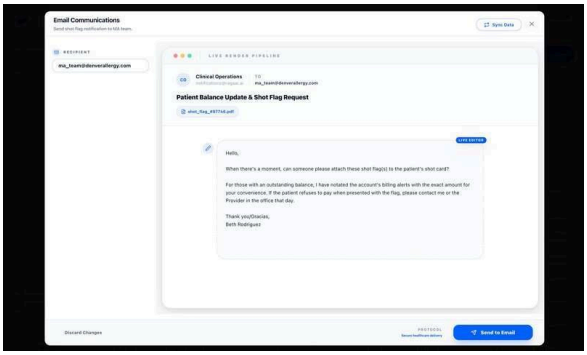
Screens Developed:



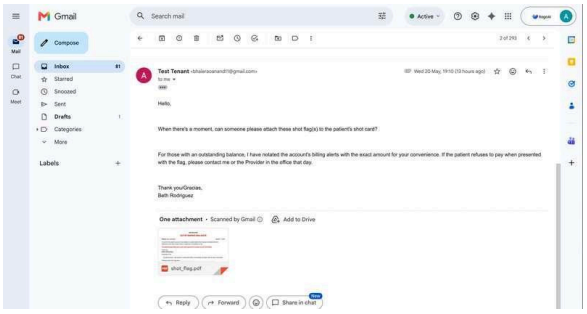
Collections Agent — Overview Dashboard (Live API)



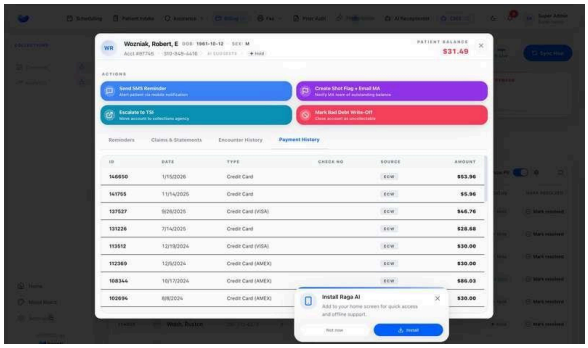
Patient Detail Dialog — 4 Action Buttons (All API Connected)



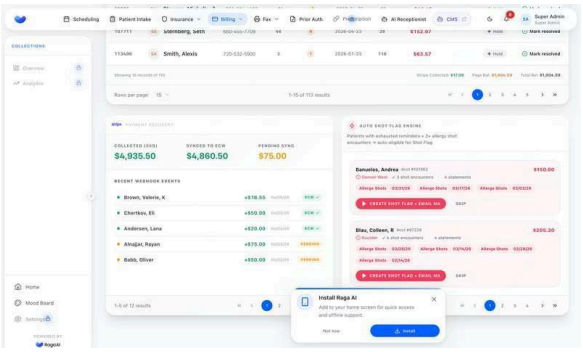
Email Communications — Live Render Pipeline with PDF



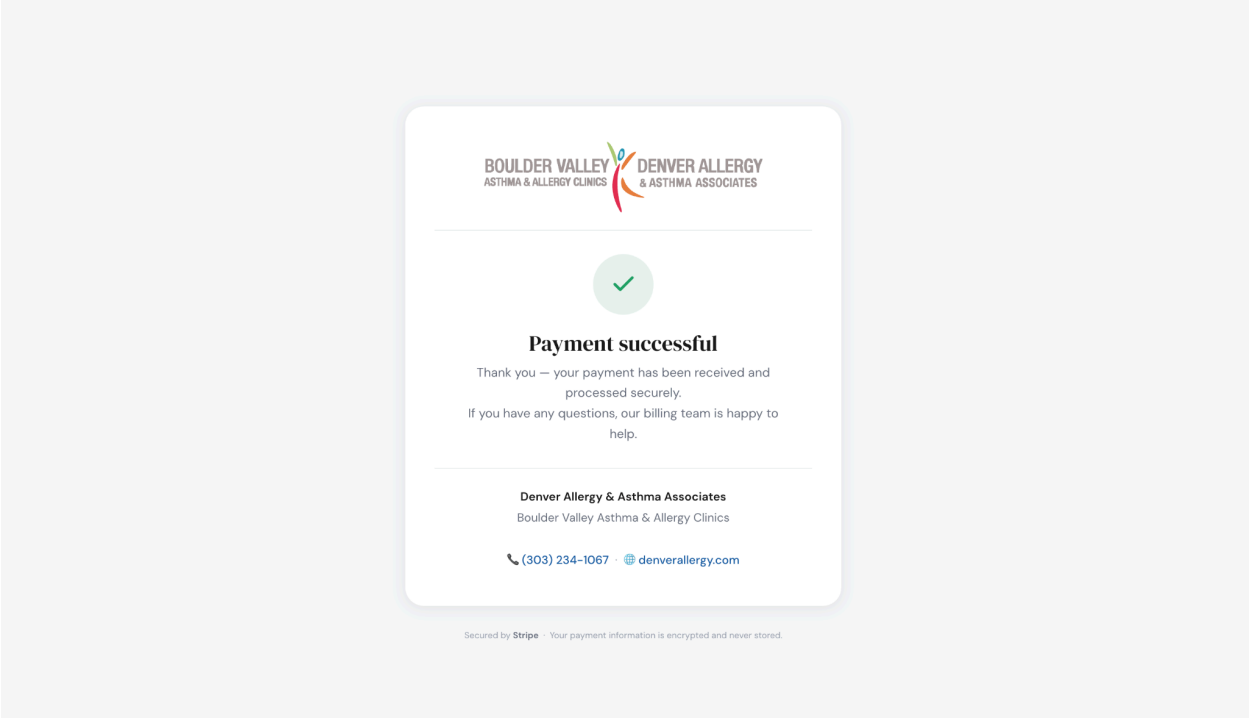
Shot Flag Email Successfully Delivered — Gmail Proof ✓



Patient Payment History Tab (Live API)



Payment Recovery & Auto Shot Flag Engine (In Progress — Dummy Data)



Payment successful

3.2 CMS — ClinicalOps CMS

Period: March – May 2026

Contributed to the ClinicalOps CMS — RagaAI's content and operations management platform. The work spanned two major modules: Public Reviews (Patient Reputation Hub) and Support Tickets, both fully integrated with live backend APIs.

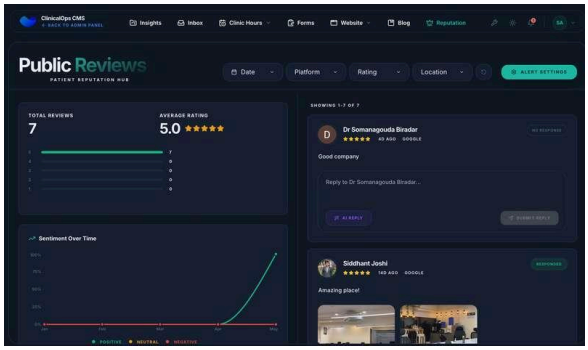
Link: https://clinicalops.ragaai.ai/cms/reputation/public-review

3.2.1 Public Reviews — Patient Reputation Hub

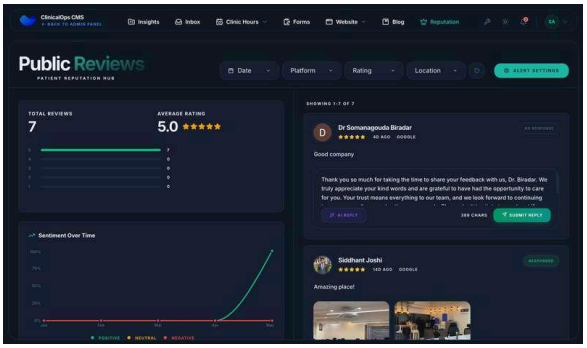
Key Contributions:

- Built the Public Reviews dashboard displaying Total Reviews (7) and Average Rating (5.0 ★★★★★) pulled live from the Google Reviews API.
- Integrated a Sentiment Over Time chart showing positive, neutral, and negative review trends across Jan–May 2026, rendered with live data.
- Implemented a Ratings by Site breakdown (Google: 5.0/5) and Reviews by Site count (Google: 7 Reviews) using live API data.
- Built individual review cards showing reviewer name, platform (Google), date, star rating, review text, response status (Responded / No Response), and profile photos.
- Integrated an AI Reply feature — clicking 'AI Reply' generates a contextual, professional response (389 chars) populated directly into the reply input field using the AI API, ready for submission.
- Implemented the Submit Reply flow — allowing admins to post replies directly to Google reviews from within the CMS dashboard.
- Built a Review Alert Configuration dialog allowing admins to toggle email alerts on/off, set an alert threshold by star rating (1★–5★ slider), and manage up to 5 notification recipient email addresses.
- Verified successful alert delivery — the review alert system correctly sends email notifications to configured recipients when the threshold condition is met.
- Added filter controls for Date, Platform, Rating, and Location dropdowns with a reset option for flexible review management.

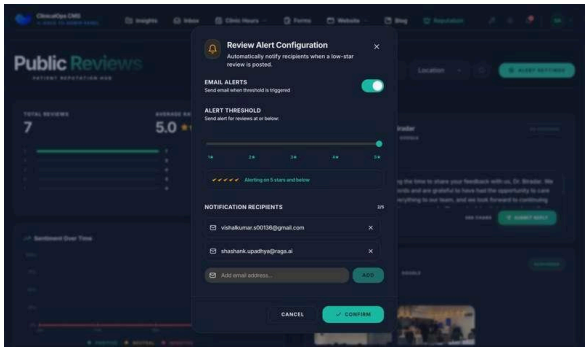
Screens Developed:



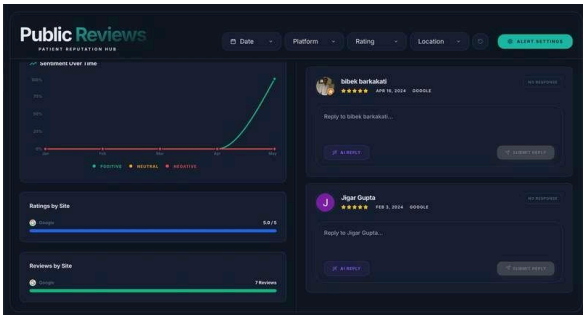
Public Reviews — Live Dashboard & Sentiment Chart



AI Reply Generated & Ready to Submit (Live API)



Review Alert Configuration — Email Threshold & Recipients



Ratings by Site & Reviews by Site — Live Google Data

3.2.2 Support Tickets Module

Built a full-featured Support Tickets system within the ClinicalOps CMS — a Kanban-style ticket management board for tracking ideas, feedback, and design concepts across To Do, In Progress, and Resolved statuses. The module includes complete API integration with verified email notification delivery on ticket creation.

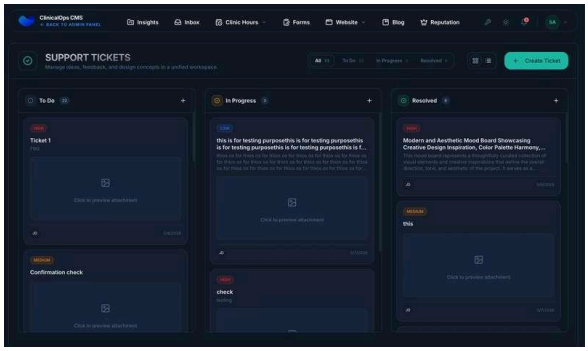
Link: https://clinicalops.ragaai.ai/cms/support-tickets

Key Contributions:

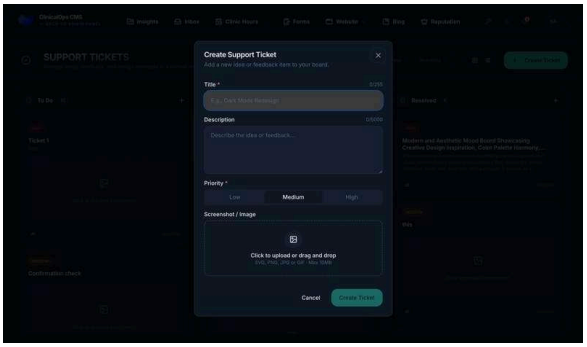
- Designed and built the Support Tickets Kanban board with three columns — To Do (22), In Progress (3), and Resolved (8) — with live ticket counts fetched from the API.
- Implemented ticket cards showing title, description, priority badge (High / Medium / Low), assignee avatar, date, and a screenshot/attachment preview area.
- Built the Create Support Ticket dialog with Title (0/255), Description (0/5000), Priority selector (Low / Medium / High toggle), and Screenshot/Image upload (SVG, PNG, JPG, GIF — Max 10MB drag-and-drop).
- Integrated the ticket creation API — on form submission, a new ticket is created in the backend and instantly appears on the Kanban board under the correct status column.
- Built the Ticket Detail view showing full title, description, status badge (Resolved), priority level (Medium), submission timeline, system reference ID, assignee, attachments & evidence section, and a Resolution Tip panel.

- Implemented Change Status controls on the detail page — admins can move tickets between To Do, In Progress, and Resolved directly from the detail view.
- Integrated email notification on ticket creation — when a ticket is created, an automated email is triggered with full ticket details including Support ID, Tenant Name, Priority, Created By, Created At, Title, Description, and a direct link to view and manage the ticket in the CMS dashboard.
- Verified end-to-end email delivery — the support ticket notification email was successfully received in Gmail with all ticket metadata and a clickable CMS deep link (clinicalops.ragaai.ai/cms/support-tickets/...).
- Supported both Grid and List view modes on the Kanban board for flexible ticket browsing.

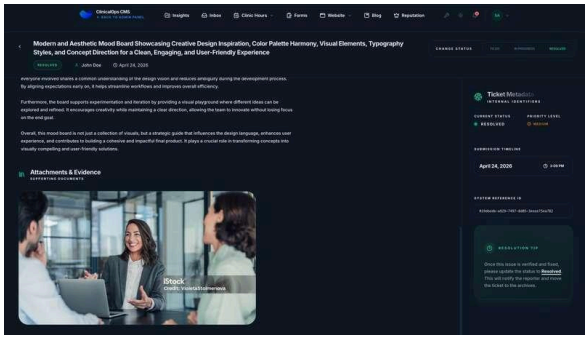
Screens Developed:



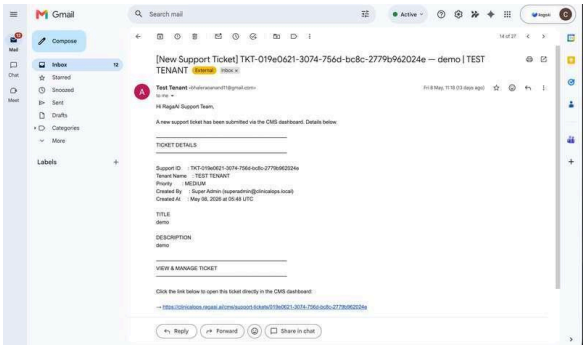
Support Tickets — Kanban Board (Live API)



Create Support Ticket — Dialog with Priority & Upload



Ticket Detail View — Status, Metadata & Attachments



Ticket Creation Email Successfully Delivered — Gmail Proof ✓

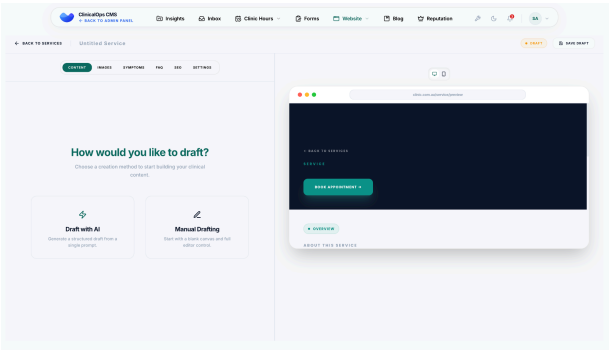
3.2.3 CMS Website Development & Frontend Contributions

Worked on the frontend development of the ClinicalOps CMS platform by building and enhancing multiple dashboard sections, workflow modules, and AI-integrated features.

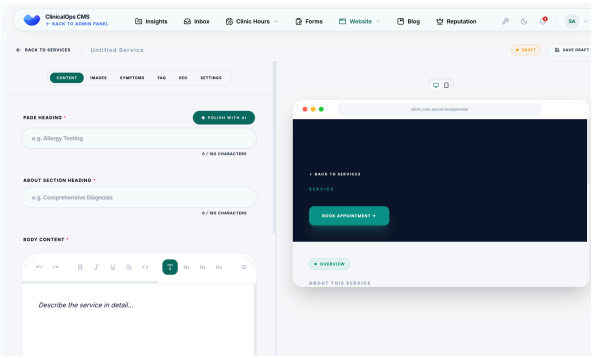
Link: https://clinicalops.ragaai.ai/cms/website/services/new

Key Contributions:

- Developed responsive frontend modules using React.js, TypeScript, and Tailwind CSS.
- Built reusable UI components, forms, modals, tables, and dashboard layouts for CMS workflows.
- Integrated APIs for real-time data fetching, updates, status management, and workflow operations.
- Worked on manual workflow sections for task handling, ticket management, and admin operations.
- Contributed to AI-powered CMS sections by integrating AI APIs and dynamic workflow interfaces.
- Implemented validations, loading states, error handling, and responsive UI behavior.
- Performed frontend bug fixes, UI enhancements, and optimization across multiple CMS modules.
- Collaborated with backend and QA teams to deliver production-ready features and smooth user experience.



CMS website — create with ai or manually



CMS website — create with ai or manually

3.3 Denver Allergy

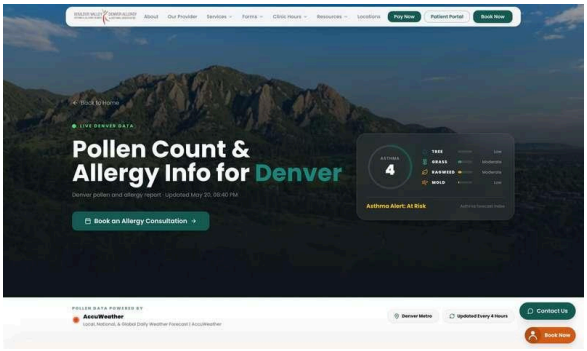
Period: March – May 2026

Contributed to the Denver Allergy & Asthma Associates website by designing a unique Pollen Count & Allergy Info page and integrating live allergy forecast data via the AccuWeather API. The work covered real-time asthma, ragweed, tree, grass, and mold pollen tracking with a custom-built UI.

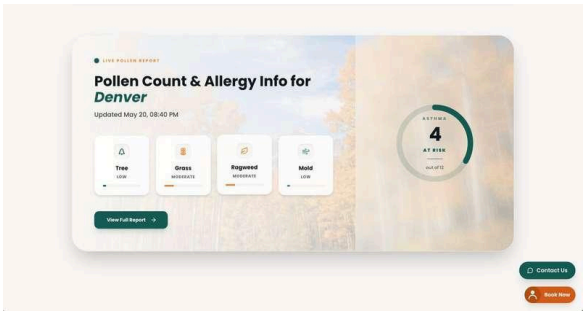
Key Contributions:

- Designed and built the Pollen Count & Allergy Info page for Denver with a full-screen hero section, live asthma index indicator, and pollen level breakdown (Tree, Grass, Ragweed, Mold).
- Integrated AccuWeather API to fetch and display real-time pollen and asthma forecast data updated every 4 hours for the Denver Metro area.
- Built a 5-Day Allergy Forecast table with circular index indicators for Asthma, Ragweed, Tree, Grass, Mold, AQI, and UV Index per day.
- Developed a reusable Live Pollen Report widget (embeddable card) showing allergy category cards with severity levels and an asthma risk gauge.
- Designed a Data Partnership section explaining the AccuWeather integration with Global Network and Real-Time Updates feature cards.
- Created a unique, visually consistent design with nature-inspired backgrounds, teal/green brand colors, and smooth UI components across all sections.

Screens Developed:



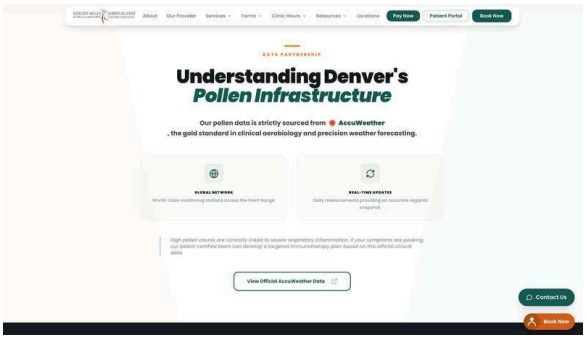
Pollen Count & Allergy Info — Hero Page



Live Pollen Report Widget



5-Day Allergy Forecast Table



AccuWeather Data Partnership Section

3.3.1 Performance Optimization

Conducted a full frontend performance audit using Google Lighthouse and implemented optimizations to improve the site performance score from 50 → 75 on the live production deployment (denver-allergy.ragaai.ai):

- Implemented lazy loading for all images and below-the-fold components using React lazy() and Intersection Observer, significantly reducing initial page load time.
- Compressed and converted all images to optimized formats — reduced First Contentful Paint from 14.2s → 2.3s and Largest Contentful Paint from 33.0s → 2.3s.
- Applied code splitting and dynamic imports to reduce JavaScript bundle size and improve Time to Interactive.
- Optimized font loading using font-display: swap to eliminate render-blocking caused by web fonts.
- Removed unused CSS and deferred non-critical scripts to improve overall page load efficiency.

3.3.2 Accessibility Improvements

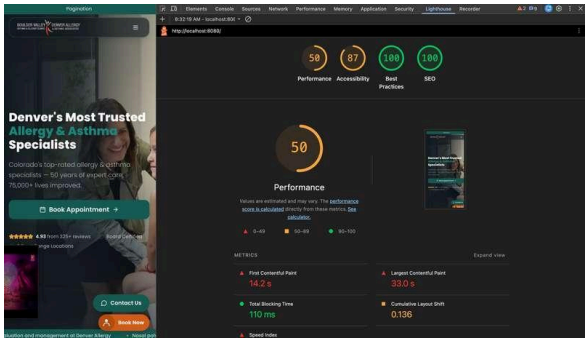
Alongside performance, reviewed and improved the site's accessibility following WCAG standards, achieving an Accessibility score of 85/100 on the live deployment:

- Added descriptive alt text to all images and icon elements for screen reader compatibility.
- Ensured all interactive elements (buttons, links, form fields) have proper ARIA labels and roles.
- Verified sufficient color contrast ratios across all text and background combinations.
- Improved keyboard navigation flow across all pages and modal components.
- Maintained semantic HTML structure using correct heading hierarchy (h1 → h2 → h3) throughout.

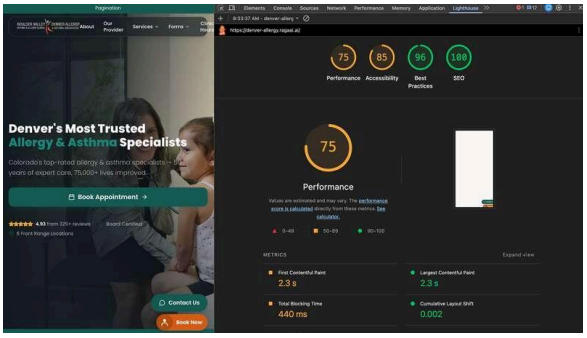
Lighthouse Scores — Before vs After:

Metric	Before (localhost)	After (Production)
Performance	50 / 100	75 / 100 (+25)
Accessibility	87 / 100	85 / 100
Best Practices	100 / 100	96 / 100
SEO	100 / 100	100 / 100
First Contentful Paint	14.2 s	2.3 s
Largest Contentful Paint	33.0 s	2.3 s
Total Blocking Time	110 ms	440 ms
Cumulative Layout Shift	0.136	0.002

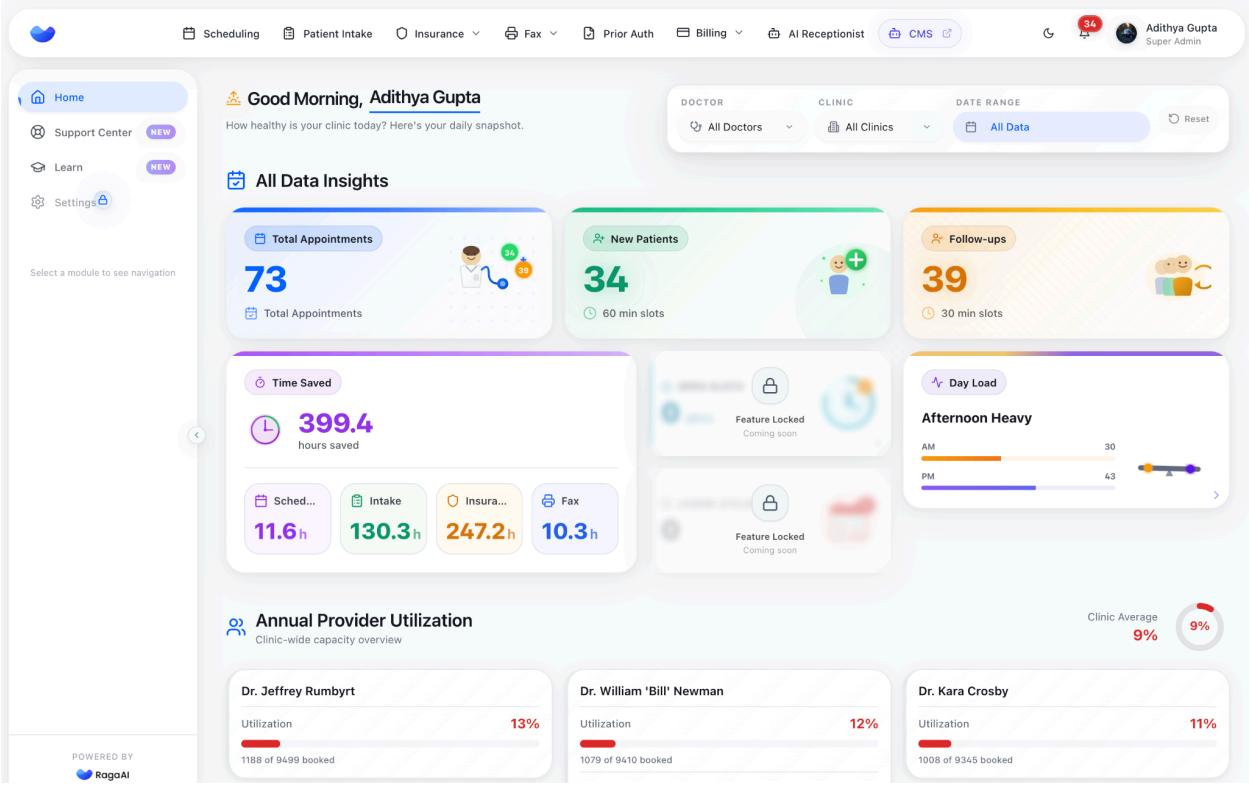
Lighthouse Audit — Before & After Screenshots:

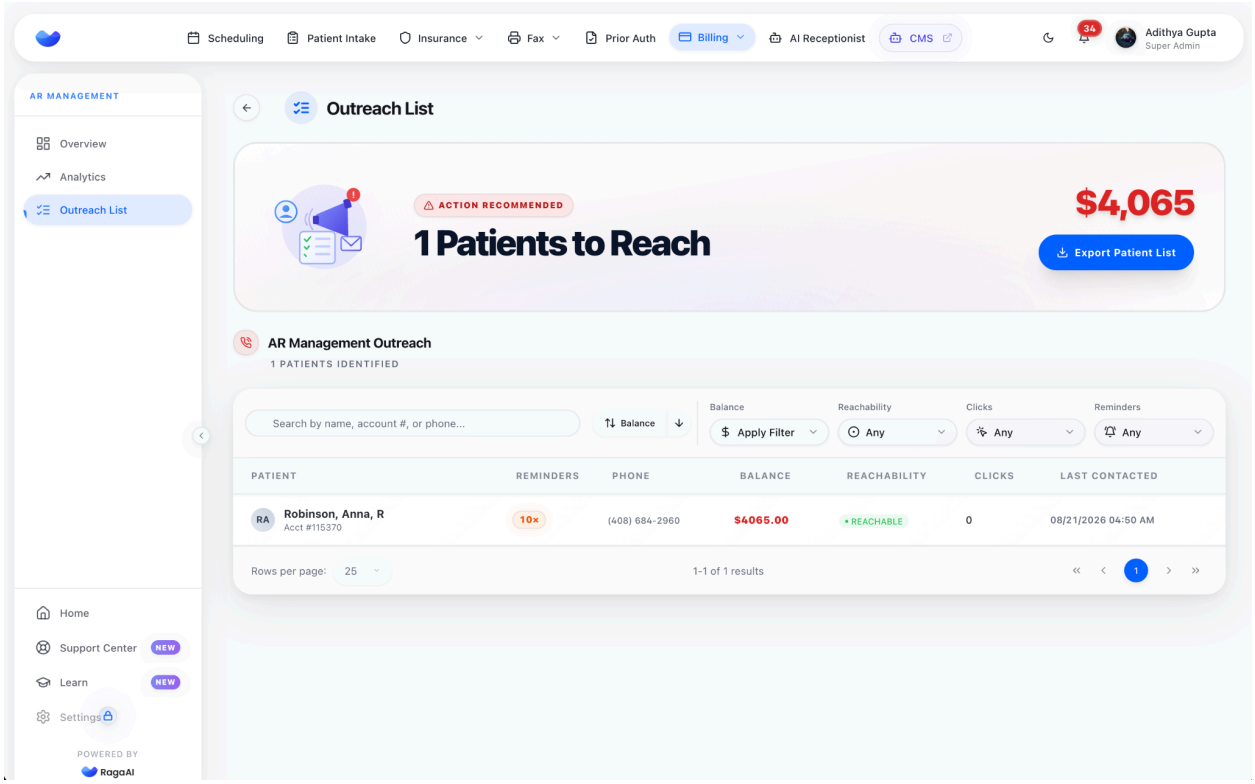


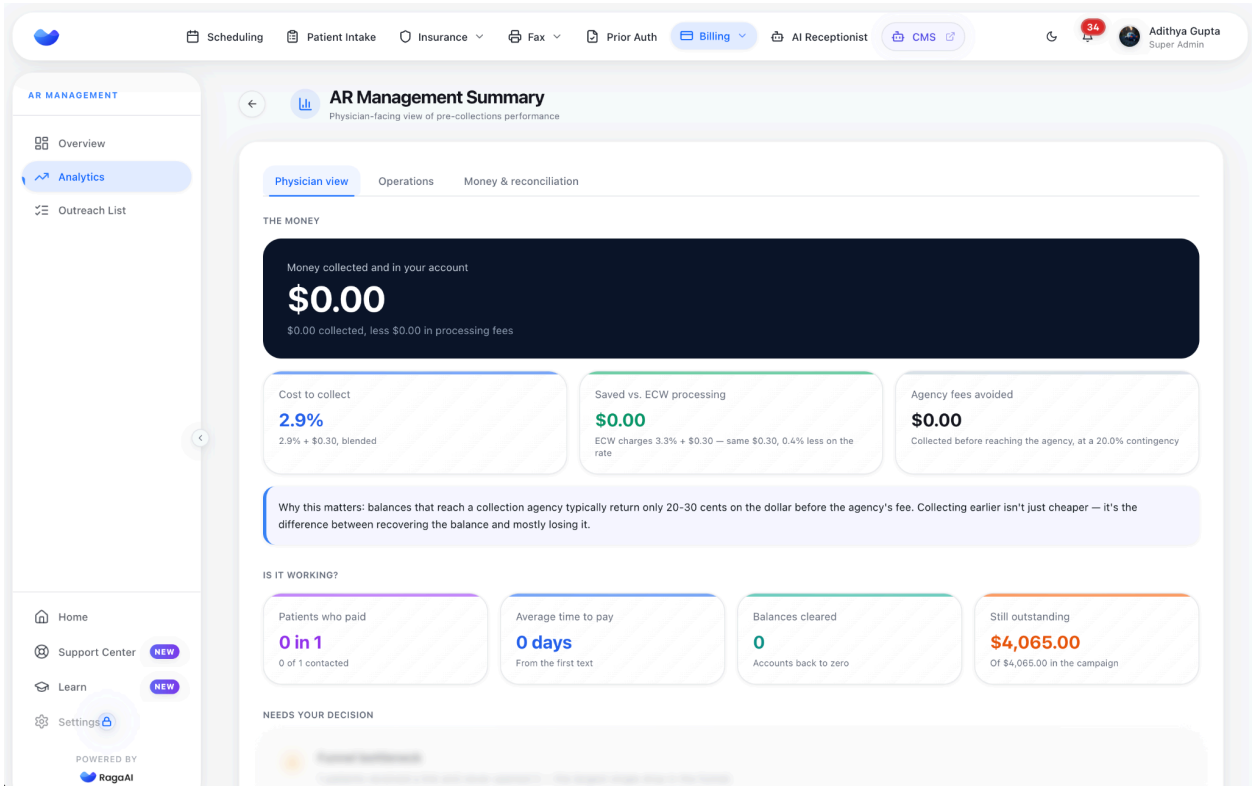
Before — Perf: 50 | Access: 87 | BP: 100 | SEO: 100

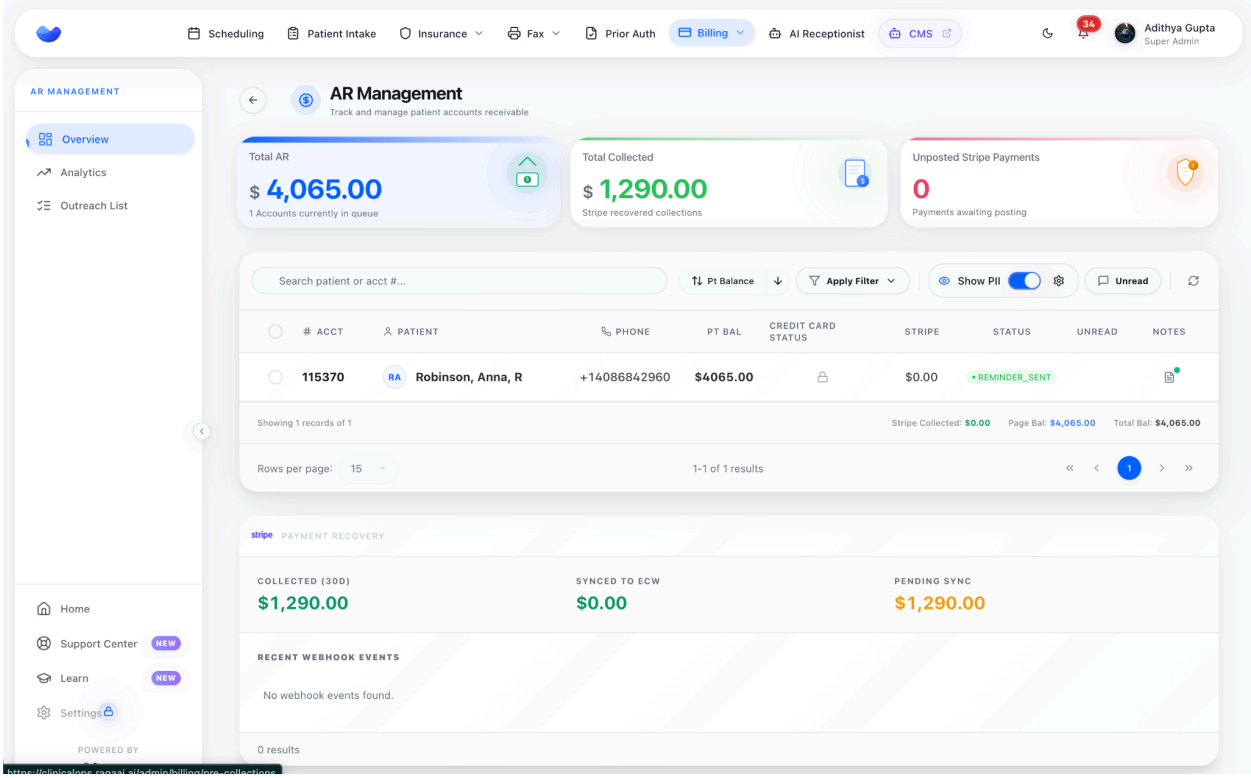


After — Perf: 75 | Access: 85 | BP: 96 | SEO: 100




Scheduling
Patient Intake
Insurance
Fax
Prior Auth
Billing
AI Receptionist
CMS
34
Adithya Gupta
Super Admin
AR MANAGEMENT
Overview
Analytics
Outreach List
Outreach List
ACTION RECOMMENDED
1 Patients to Reach
$4,065
Export Patient List
AR Management Outreach
1 PATIENTS IDENTIFIED
Search by name, account #, or phone...
Balance
Apply Filter
Any
Any
Any
PATIENT
REMINDERS
PHONE
BALANCE
REACHABILITY
CLICKS
LAST CONTACTED
Robinson, Anna, R
Acct #115370
10×
(408) 684-2960
$4065.00
REACHABLE
0
08/21/2026 04:50 AM
Rows per page: 25
1-1 of 1 results
Home
Support Center
NEW
Learn
NEW
Settings
POWERED BY
RagaAI


Scheduling
Patient Intake
Insurance
Fax
Prior Auth
Billing
AI Receptionist
CMS
34
Adithya Gupta
Super Admin
AR MANAGEMENT
Overview
Analytics
Outreach List
AR Management Summary
Physician-facing view of pre-collections performance
Physician view
Operations
Money & reconciliation
THE MONEY
Money collected and in your account
$0.00
$0.00 collected, less $0.00 in processing fees
Cost to collect
2.9%
2.9% + $0.30, blended
Saved vs. ECW processing
$0.00
ECW charges 3.3% + $0.30 — same $0.30, 0.4% less on the rate
Agency fees avoided
$0.00
Collected before reaching the agency, at a 20.0% contingency
Why this matters: balances that reach a collection agency typically return only 20-30 cents on the dollar before the agency's fee. Collecting earlier isn't just cheaper — it's the difference between recovering the balance and mostly losing it.
IS IT WORKING?
Patients who paid
0 in 1
0 of 1 contacted
Average time to pay
0 days
From the first text
Balances cleared
0
Accounts back to zero
Still outstanding
$4,065.00
Of $4,065.00 in the campaign
NEEDS YOUR DECISION
Home
Support Center
NEW
Learn
NEW
Settings
POWERED BY
RagaAI



3.4 Additional Task — Raga-admin UI Design

Repository: github.com/adityaragaai/Raga-admin

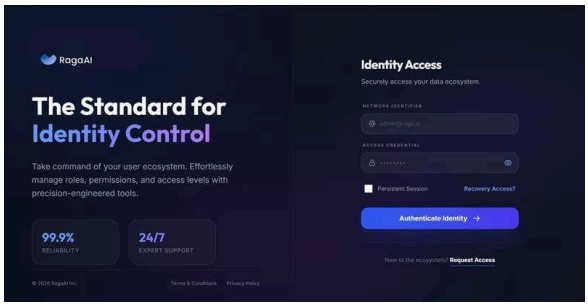
As an additional contribution, a fully designed admin UI was built for RagaAI covering identity access, customer management, and user profile management. The following screens were developed:

Login Page — Identity Access:

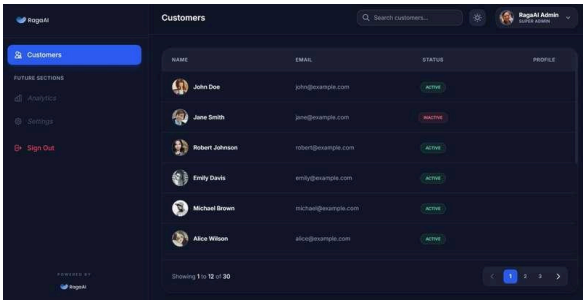
A premium dark-themed login screen featuring network identifier and access credential inputs, persistent session toggle, recovery access link, and an Authenticate Identity CTA. Includes reliability (99.9%) and support (24/7) metric cards.

Customers Dashboard & User Profile Drawer:

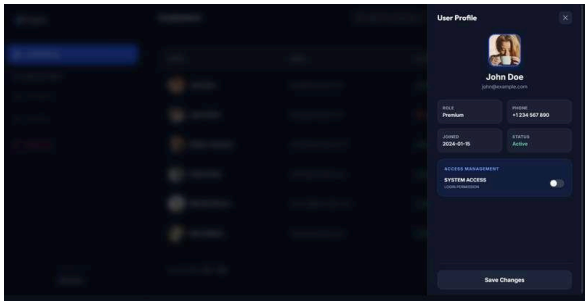
A paginated customer management dashboard with live search, status badges, and avatar display. Paired with a slide-over User Profile drawer showing role, contact info, join date, status, and an Access Management toggle.



Login — Identity Access



Customers Dashboard



User Profile Drawer

Key Contributions:

- Designed and implemented a dark-themed, premium UI consistent with the RagaAI brand identity.
- Built the Identity Access login page with secure credential fields, persistent session, and recovery access flow.
- Developed a paginated Customers dashboard with live search, status badges, and avatar display.
- Implemented a slide-over User Profile drawer with role, status, contact info, and access management toggle.
- Ensured consistent design language, spacing, and component reuse across all screens.

3.5 Bug Fixes & Maintenance Contributions

Period: March – May 2026

In addition to feature development, actively contributed to daily bug fixing, maintenance, and UI/UX improvements across multiple RagaAI products including Clinical Ops, CMS, and Denver Allergy platforms.

Key Contributions:

- Resolved multiple frontend bugs and UI inconsistencies reported across Clinical Ops, CMS, and Denver Allergy projects on a day-to-day basis.

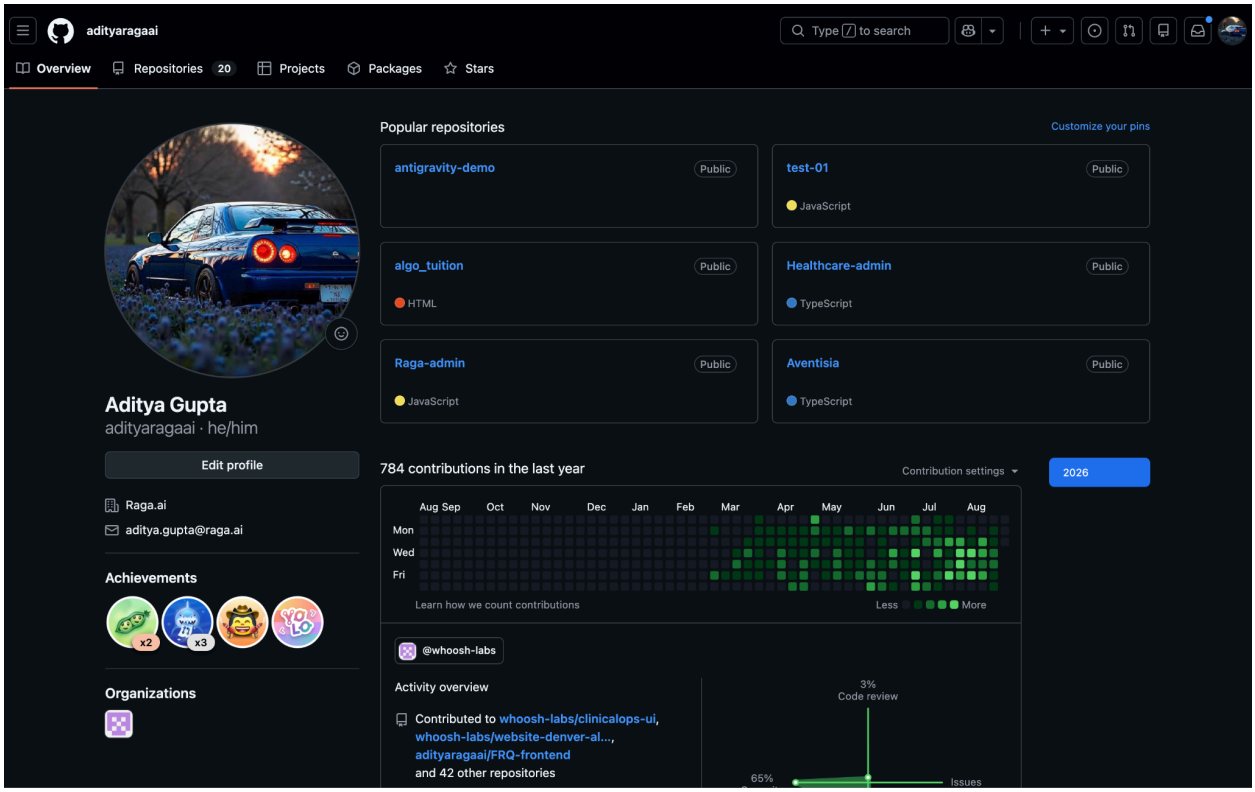
- Fixed API integration issues, data rendering problems, loading state bugs, and responsive layout inconsistencies.
- Worked on debugging production and staging issues related to forms, tables, dialogs, filters, pagination, and real-time data synchronization.
- Improved component stability and reduced UI breakages by handling edge cases and fallback states.
- Collaborated with team members to reproduce, investigate, and resolve bugs raised internally by developers, QA, and product teams.
- Contributed fixes not only for self-developed modules but also for issues across other developers' implementations.
- Performed regression testing after fixes to ensure stability across interconnected modules and workflows.
- Maintained code quality and consistency while working within large-scale shared codebases.

- Fixed an issue in conditional radio branching where components from the first branch were incorrectly carried over into newly created branches when duplicate branch names were assigned.
- Resolved hyperlink validation behavior where `https://` was prefilled by default, causing invalid URLs when users pasted complete links without removing the existing prefix.
- Investigated, reproduced, and verified fixes for branch handling and hyperlink generation workflows to ensure stable user experience and accurate data handling.
-

4. Declaration

I, Aditya Gupta, confirm that all work referenced in this document — including feature development, UI implementation, API integrations, performance optimization, accessibility improvements, bug fixes, and maintenance contributions — was independently developed, contributed, and committed by me to the GitHub repositories and projects listed above during my internship phase at Raga.ai.

The code, architectural decisions, debugging efforts, and frontend implementations represent my personal contribution across Clinical Ops, CMS, Denver Allergy, and supporting internal platforms.


adityaragaai
Type / to search
Overview
Repositories 20
Projects
Packages
Stars
Aditya Gupta
adityaragaai · he/him
Edit profile
Raga.ai
aditya.gupta@raga.ai
Achievements
x2
x3
Organizations
Popular repositories
Customize your pins
antigravity-demo
Public
test-01
Public
JavaScript
algo_tuition
Public
HTML
Healthcare-admin
Public
TypeScript
Raga-admin
Public
JavaScript
Aventisia
Public
TypeScript
784 contributions in the last year
Contribution settings
2026
Aug Sep Oct Nov Dec Jan Feb Mar Apr May Jun Jul Aug
Mon
Wed
Fri
Learn how we count contributions
Less More
@whoosh-labs
Activity overview
Contributed to whoosh-labs/clinicalops-ui, whoosh-labs/website-denver-al..., adityaragaai/FRQ-frontend and 42 other repositories
3%
Code review
65%
Issues